Lab 9 - Programs on Matrix Using CUDA (cont)

Q1.cu

```
#include <stdio.h>
#include <cuda_runtime.h>

#define N 3

__global__ void spmv_csr(float *data, int *col_index, int *row_ptr, float *x, float *y, int num_rows) {
int row = blockIdx.x * blockDim.x + threadIdx.x;
if (row < num_rows) {
float sum = 0.0f;
for (int j = row_ptr[row]; j < row_ptr[row + 1]; j++) {
sum += data[j] * x[col_index[j]];
}
y[row] = sum;
}
}

int main() {
float h_data[] = {1, 2, 3, 4, 5, 6};
int h_col_index[] = {0, 2, 0, 1, 2, 1};
int h_row_ptr[] = {0, 2, 5, 6};
float h_x[] = {1, 2, 3};
float h_y[N] = {0};

int nnz = 6;
int num_rows = N;

float *d_data, *d_x, *d_y;
int *d_col_index, *d_row_ptr;

cudaMalloc(&d_data, nnz * sizeof(float));
cudaMalloc(&d_col_index, nnz * sizeof(int));
cudaMalloc(&d_row_ptr, (num_rows + 1) * sizeof(int));
cudaMalloc(&d_x, N * sizeof(float));
cudaMalloc(&d_y, N * sizeof(float));

cudaMemcpy(d_data, h_data, nnz * sizeof(float), cudaMemcpyHostToDevice);
cudaMemcpy(d_col_index, h_col_index, nnz * sizeof(int), cudaMemcpyHostToDevice);
cudaMemcpy(d_row_ptr, h_row_ptr, (num_rows + 1) * sizeof(int), cudaMemcpyHostToDevice);
cudaMemcpy(d_x, h_x, N * sizeof(float), cudaMemcpyHostToDevice);
```

```
int threadsPerBlock = 256;
int blocksPerGrid = (num_rows + threadsPerBlock - 1) / threadsPerBlock;

spmv_csr<<<blocksPerGrid, threadsPerBlock>>>(d_data, d_col_index, d_row_ptr, d_x, d_y, num_rows);

cudaMemcpy(h_y, d_y, N * sizeof(float), cudaMemcpyDeviceToHost);

printf("Result vector y:\n");
for (int i = 0; i < N; i++) {
printf("y[%d] = %.2f\n", i, h_y[i]);
}

cudaFree(d_data);
cudaFree(d_col_index);
cudaFree(d_row_ptr);
cudaFree(d_x);
cudaFree(d_y);

return 0;
}
```

Output:

```
6CSEC1@debian:~/Documents/230905152_PPL/L9$ nvcc -o abc q1.cu
6CSEC1@debian:~/Documents/230905152_PPL/L9$ ./abc
Result vector y:
y[0] = 7.00
y[1] = 26.00
y[2] = 12.00
```

Q2.cu

```
#include <stdio.h>
#include <cuda_runtime.h>

#define M 3
#define N 3

__global__ void transform(float *A, int m, int n) {
int row = blockIdx.y * blockDim.y + threadIdx.y;
int col = blockIdx.x * blockDim.x + threadIdx.x;

if (row < m && col < n) {
int idx = row * n + col;
float val = A[idx];
```

```
int power = row + 1;
float result = 1.0f;
for (int i = 0; i < power; i++) {
result *= val;
}
A[idx] = result;
}
}

int main() {
float h_A[M][N] = {
{1, 2, 3},
{2, 3, 4},
{2, 3, 4}
};

printf("Original matrix:\n");
for (int i = 0; i < M; i++) {
for (int j = 0; j < N; j++)
printf("%.2f ", h_A[i][j]);
printf("\n");
}

float *d_A;
cudaMalloc(&d_A, M * N * sizeof(float));
cudaMemcpy(d_A, h_A, M * N * sizeof(float), cudaMemcpyHostToDevice);

dim3 threadsPerBlock(16, 16);
dim3 blocksPerGrid((N + 15) / 16, (M + 15) / 16);

transform<<<blocksPerGrid, threadsPerBlock>>>(d_A, M, N);

cudaMemcpy(h_A, d_A, M * N * sizeof(float), cudaMemcpyDeviceToHost);

printf("\nTransformed matrix:\n");
for (int i = 0; i < M; i++) {
for (int j = 0; j < N; j++)
printf("%.2f ", h_A[i][j]);
printf("\n");
}

cudaFree(d_A);
return 0;
}
```

Output:

```
6CSEC1@debian:~/Documents/230905152_PPL/L9$ nvcc -o abc q2.cu
6CSEC1@debian:~/Documents/230905152_PPL/L9$ ./abc
Original matrix:
1.00 2.00 3.00
2.00 3.00 4.00
2.00 3.00 4.00

Transformed matrix:
1.00 2.00 3.00
4.00 9.00 16.00
8.00 27.00 64.00
```

Q3.cu

```
#include <stdio.h>
#include <cuda_runtime.h>

#define M 4
#define N 4

__global__ void ones_complement(int *A, int *B, int m, int n) {
int row = blockIdx.y * blockDim.y + threadIdx.y;
int col = blockIdx.x * blockDim.x + threadIdx.x;

if (row < m && col < n) {
int idx = row * n + col;
if (row == 0 || row == m-1 || col == 0 || col == n-1) {
B[idx] = A[idx];
} else {
B[idx] = ~A[idx];
}
}
}

void print_binary(int n) {
if (n == 0) { printf("0"); return; }
int bits[32], count = 0;
unsigned int un = (unsigned int)n;
while (un > 0) {
bits[count++] = un & 1;
un >>= 1;
}
```

```
for (int i = count - 1; i >= 0; i--)
printf("%d", bits[i]);
}

int main() {
int h_A[M][N] = {
{1, 2, 3, 4},
{6, 5, 8, 3},
{2, 4, 10, 1},
{9, 1, 2, 5}
};

int h_B[M][N] = {0};

printf("Matrix A:\n");
for (int i = 0; i < M; i++) {
for (int j = 0; j < N; j++)
printf("%4d ", h_A[i][j]);
printf("\n");
}

int *d_A, *d_B;
cudaMalloc(&d_A, M * N * sizeof(int));
cudaMalloc(&d_B, M * N * sizeof(int));

cudaMemcpy(d_A, h_A, M * N * sizeof(int), cudaMemcpyHostToDevice);

dim3 threadsPerBlock(16, 16);
dim3 blocksPerGrid((N + 15) / 16, (M + 15) / 16);

ones_complement<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, M, N);

cudaMemcpy(h_B, d_B, M * N * sizeof(int), cudaMemcpyDeviceToHost);

printf("\nMatrix B (non-border shown in binary):\n");
for (int i = 0; i < M; i++) {
for (int j = 0; j < N; j++) {
if (i == 0 || i == M-1 || j == 0 || j == N-1)
printf("%4d ", h_B[i][j]);
else {
printf("%4d ", h_A[i][j] ^ ((1 << (int)(log2(h_A[i][j]) + 1)) - 1));
}
}
printf("\n");
```

```
}

printf("\nMatrix B (non-border in binary format as shown in question):\n");
for (int i = 0; i < M; i++) {
for (int j = 0; j < N; j++) {
if (i == 0 || i == M-1 || j == 0 || j == N-1)
printf("%5d ", h_A[i][j]);
else {
int comp = h_A[i][j] ^ ((1 << (int)(log2(h_A[i][j]) + 1)) - 1);
printf("%5d ", comp);
printf("(");
print_binary(comp);
printf(") ");
}
}
printf("\n");
}

cudaFree(d_A);
cudaFree(d_B);
return 0;
}
```

Output:

```
6CSEC1@debian:~/Documents/230905152_PPL/L9$ nvcc -o abc q3.cu
6CSEC1@debian:~/Documents/230905152_PPL/L9$ ./abc
Matrix A:
    1     2     3     4
    6     5     8     3
    2     4    10     1
    9     1     2     5

Matrix B (non-border shown in binary):
    1     2     3     4
    6     2     7     3
    2     3     5     1
    9     1     2     5

Matrix B (non-border in binary format as shown in question):
     1      2      3      4
     6      2 (10)      7 (111)      3
     2      3 (11)      5 (101)      1
     9      1      2      5
```

A1.cu

```
#include <stdio.h>
#include <cuda_runtime.h>

#define M 2
#define N 3

__global__ void compute_B(int *A, int *B, int m, int n) {
int row = blockIdx.y * blockDim.y + threadIdx.y;
int col = blockIdx.x * blockDim.x + threadIdx.x;

if (row < m && col < n) {
int row_sum = 0;
for (int j = 0; j < n; j++)
row_sum += A[row * n + j];

int col_sum = 0;
for (int i = 0; i < m; i++)
col_sum += A[i * n + col];

B[row * n + col] = row_sum + col_sum;
}
}

int main() {
int h_A[M][N] = {
{1, 2, 3},
{4, 5, 6}
};
int h_B[M][N] = {0};

printf("Matrix A:\n");
for (int i = 0; i < M; i++) {
for (int j = 0; j < N; j++)
printf("%4d ", h_A[i][j]);
printf("\n");
}

int *d_A, *d_B;
cudaMalloc(&d_A, M * N * sizeof(int));
cudaMalloc(&d_B, M * N * sizeof(int));

cudaMemcpy(d_A, h_A, M * N * sizeof(int), cudaMemcpyHostToDevice);
```

```
dim3 threadsPerBlock(16, 16);
dim3 blocksPerGrid((N + 15) / 16, (M + 15) / 16);

compute_B<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, M, N);

cudaMemcpy(h_B, d_B, M * N * sizeof(int), cudaMemcpyDeviceToHost);

printf("\nMatrix B:\n");
for (int i = 0; i < M; i++) {
for (int j = 0; j < N; j++)
printf("%4d ", h_B[i][j]);
printf("\n");
}

cudaFree(d_A);
cudaFree(d_B);
return 0;
}
```

Output:

```
6CSEC1@debian:~/Documents/230905152_PPL/L9$ nvcc -o abc a1.cu
6CSEC1@debian:~/Documents/230905152_PPL/L9$ ./abc
Matrix A:
   1    2    3
   4    5    6

Matrix B:
  11   13   15
  20   22   24
```

A2.cu

```
#include <stdio.h>
#include <string.h>
#include <cuda_runtime.h>

#define M 2
#define N 4

__global__ void repeat_chars(char *A, int *B, char *STR, int *offsets, int m, int n) {
int row = blockIdx.y * blockDim.y + threadIdx.y;
int col = blockIdx.x * blockDim.x + threadIdx.x;
```

```
if (row < m && col < n) {
int idx = row * n + col;
int start = offsets[idx];
for (int k = 0; k < B[idx]; k++) {
STR[start + k] = A[idx];
}
}
}

int main() {
char h_A[M][N] = {
{'p', 'C', 'a', 'P'},
{'e', 'X', 'a', 'M'}
};
int h_B[M][N] = {
{1, 2, 4, 3},
{2, 4, 3, 2}
};

int h_offsets[M * N];
int total = 0;
for (int i = 0; i < M; i++) {
for (int j = 0; j < N; j++) {
h_offsets[i * N + j] = total;
total += h_B[i][j];
}
}

printf("Total output length: %d\n", total);

char *d_A, *d_STR;
int *d_B, *d_offsets;

cudaMalloc(&d_A, M * N * sizeof(char));
cudaMalloc(&d_B, M * N * sizeof(int));
cudaMalloc(&d_offsets, M * N * sizeof(int));
cudaMalloc(&d_STR, total * sizeof(char));

cudaMemcpy(d_A, h_A, M * N * sizeof(char), cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B, M * N * sizeof(int), cudaMemcpyHostToDevice);
cudaMemcpy(d_offsets, h_offsets, M * N * sizeof(int), cudaMemcpyHostToDevice);

dim3 threadsPerBlock(16, 16);
dim3 blocksPerGrid((N + 15) / 16, (M + 15) / 16);
```

```
repeat_chars<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_STR, d_offsets, M, N);

char *h_STR = (char *)malloc((total + 1) * sizeof(char));
cudaMemcpy(h_STR, d_STR, total * sizeof(char), cudaMemcpyDeviceToHost);
h_STR[total] = '\0';

printf("Output String STR: %s\n", h_STR);

free(h_STR);
cudaFree(d_A);
cudaFree(d_B);
cudaFree(d_offsets);
cudaFree(d_STR);
return 0;
}
```

Output:

```
6CSEC1@debian:~/Documents/230905152_PPL/L9$ nvcc -o abc a2.cu
6CSEC1@debian:~/Documents/230905152_PPL/L9$ ./abc
Total output length: 21
Output String STR: pCCaaaaPPPeeXXXXaaaMM
```